SOFTWARE ENGINEERING – UNIT 4

Complete Exam Preparation Notes (6-Mark Answers)

Your Unit 4 syllabus contains three major sections:

• **UML Diagrams** – Activity, State Chart, Component and Deployment Diagrams.

• **Object-Oriented Methodologies** – Unified Methodology, Rumbaugh Methodology and Booch Methodology.

• **Software Testing Strategies** – Testing approaches, White-Box Testing, Basis Path Testing, Black-Box Testing, Validation Testing and System Testing.

I'll explain each topic in simple language with definitions, important points, examples and diagrams wherever required. The answers are structured so you can write them directly in your semester examination.

PART 1: UML DIAGRAMS

1. Activity Diagram

Definition

An Activity Diagram is a UML behavioral diagram that represents the workflow or sequence of activities performed in a system.

It shows how different activities are executed from the beginning to the end of a process.

In simple words: It represents the step-by-step flow of activities in a system.

Symbols Used in Activity Diagram

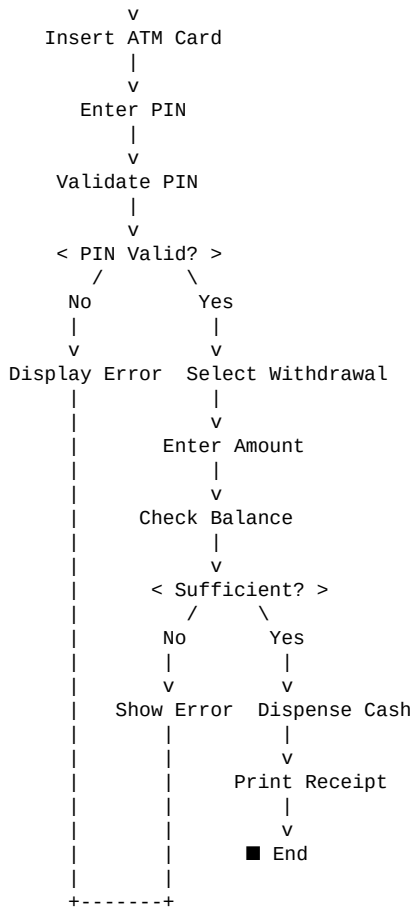
Symbol	Meaning
Filled circle (●)	Initial state
Rounded rectangle	Activity or action
Diamond (■)	Decision or condition
Arrow (→)	Flow of control
Thick horizontal/vertical line	Fork or Join
Circle with border (■)	Final state

Example: ATM Cash Withdrawal

Consider a customer withdrawing money from an ATM.

Activity Diagram:

```
● Start
  |
```



Note: This is a simplified activity flow. In an actual UML diagram, use proper decision and merge nodes.

Important Components

• **Initial Node:** Represents the starting point of the activity.

• **Activity Node:** Represents an operation performed by the system.

• **Decision Node:** Represents a condition with multiple possible paths.

• **Control Flow:** Shows the direction in which activities are executed.

• **Fork and Join:** Used to represent parallel activities and synchronize them.

• **Final Node:** Represents the completion of the activity.

Applications

• Representing business workflows.

• Designing software processes.

• Modeling user activities.

• Representing parallel operations.

• Understanding complex processes.

Advantages

• Easy to understand.

• Clearly represents workflows.

• Helps identify missing activities.

• Supports parallel processing representation.

Exam tip: Activity Diagram = Workflow or flow of activities.

2. State Chart Diagram (State Machine Diagram)

Definition

A State Chart Diagram is a UML behavioral diagram that represents the different states of an object and the transitions between those states based on events.

In simple words: It shows how an object changes its condition or state when an event occurs.

Important Concepts

• **State:** A condition or situation in which an object exists.

• **Initial State:** The starting state of an object.

• **Final State:** The ending state of an object.

• **Transition:** Movement from one state to another.

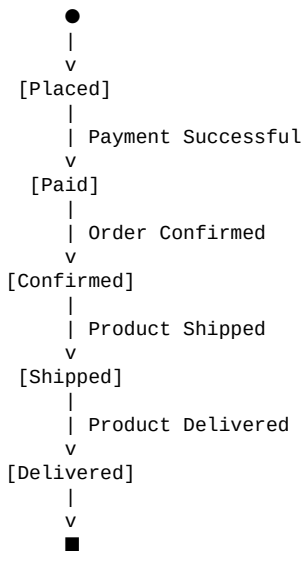
• **Event:** An occurrence that causes a state change.

• **Guard Condition:** A condition that must be satisfied for a transition to occur.

Example: Online Order Processing

An online shopping order passes through different states.

State Chart Diagram:



Additional transitions:

```
[Placed] ---- Cancel Order ----> [Cancelled]

[Paid] ------ Cancel Order ----> [Refunded]
```

Explanation

• Initially, the order is in the Placed state.

• After successful payment, it moves to Paid.

• After confirmation, it moves to Confirmed.

• When the product is shipped, it enters Shipped state.

• After delivery, it enters Delivered state.

• The customer may cancel the order before certain stages.

Applications

• ATM systems.

• Traffic light systems.

• Online shopping applications.

• Banking systems.

• Login and authentication systems.

Advantages

• Clearly represents object behavior.

• Helps understand state transitions.

• Useful for event-driven systems.

• Helps identify invalid state transitions.

Difference Between Activity and State Chart Diagrams

Activity Diagram	State Chart Diagram
Represents workflow.	Represents object states.
Focuses on activities.	Focuses on states and transitions.
Shows the flow of operations.	Shows changes in object conditions.
Used for business processes.	Used for event-driven systems.

Exam tip: State Chart Diagram = States + Events + Transitions.

3. Component Diagram

Definition

A Component Diagram is a UML structural diagram that represents the physical or logical components of a software system and the relationships between them.

A component is an independent and replaceable part of a software system.

In simple words: It shows how different software modules are organized and connected.

Components of a Component Diagram

• **Component:** Represents a software module.

• **Interface:** Defines the services provided or required by a component.

• **Dependency:** Shows that one component depends on another.

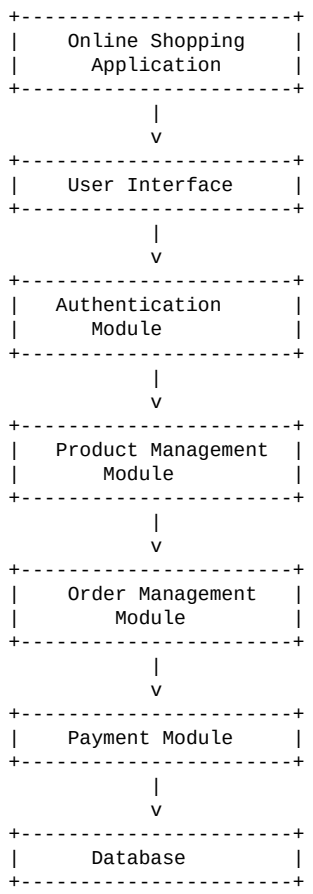
• **Port:** Represents an interaction point of a component.

• **Connector:** Connects components and interfaces.

Example: Online Shopping System

Consider an online shopping application.

Component Diagram:



Explanation

• User Interface handles user interactions.

• Authentication Module manages login and registration.

• Product Management handles product details.

• Order Management manages customer orders.

• Payment Module processes payments.

• Database stores application information.

Advantages

• Provides a high-level view of software architecture.

• Helps identify dependencies between modules.

• Supports modular development.

• Makes maintenance easier.

• Helps developers understand system organization.

Applications

• Banking applications.

• E-commerce applications.

• Hospital management systems.

• Enterprise software systems.

Exam tip: Component Diagram = Software modules and their dependencies.

4. Deployment Diagram

Definition

A Deployment Diagram is a UML structural diagram that represents the physical arrangement of hardware devices and software components in a system.

It shows where software is installed and how different hardware devices communicate.

In simple words: It explains where the software runs physically.

Important Components

• **Node:** Represents a physical or computational resource such as a computer or server.

• **Device:** Represents hardware such as a mobile phone or computer.

• **Execution Environment:** Represents the software environment where applications execute.

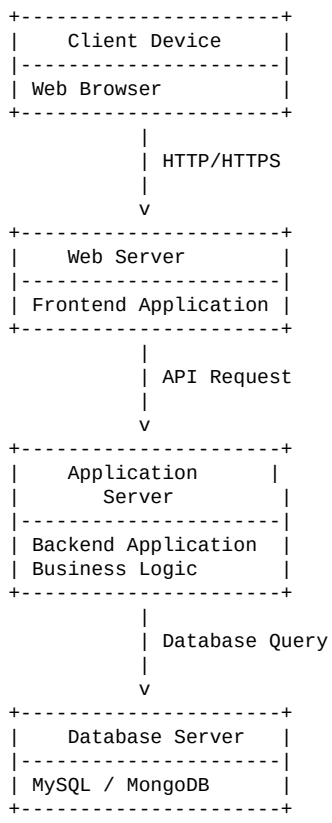
• **Artifact:** Represents a physical software file, such as an executable or JAR file.

• **Communication Path:** Represents communication between nodes.

Example: Web Application Deployment

Consider a web application accessed by a user.

Deployment Diagram:



Explanation

• Client Device: User accesses the application through a browser.

• Web Server: Handles frontend application requests.

• Application Server: Executes backend business logic.

• Database Server: Stores user and application data.

• Communication Paths: Connect the different nodes.

Advantages

• Shows physical system architecture.

• Helps in hardware and software allocation.

• Supports network planning.

• Helps identify communication requirements.

• Useful for deployment and maintenance.

Applications

• Web applications.

• Cloud-based applications.

• Distributed systems.

• Banking systems.

• Enterprise applications.

Difference Between Component and Deployment Diagrams

Component Diagram	Deployment Diagram
Represents software components.	Represents physical hardware nodes.
Focuses on software architecture.	Focuses on physical architecture.
Shows module dependencies.	Shows hardware and software deployment.
Example: Login Module.	Example: Application Server.

Important for exams:

Remember these four UML diagrams:

Diagram	Main Purpose
Activity Diagram	Workflow
State Chart Diagram	Object states
Component Diagram	Software components
Deployment Diagram	Hardware deployment

PART 2: OBJECT-ORIENTED METHODOLOGIES

5. Unified Methodology

Definition

Unified Methodology is an object-oriented software development approach that combines different software engineering techniques into a common development process.

It provides a systematic way to analyze, design, implement and test object-oriented software systems.

It is closely associated with the Unified Process (UP) and UML.

Main Characteristics

• **Use-Case Driven:** Development is based on user requirements and use cases.

• **Architecture Centric:** Focuses on developing a strong software architecture.

• **Iterative Development:** Software is developed through repeated iterations.

• **Incremental Development:** New features are added gradually.

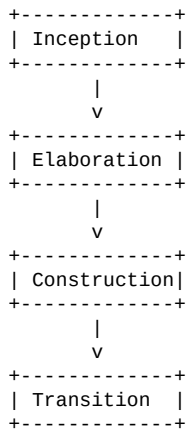
• **Risk Management:** Major risks are identified and handled early.

• **Object-Oriented:** Uses objects, classes and relationships.

Phases of Unified Process

The Unified Process contains four major phases.

Diagram:



1. Inception Phase

• Defines the project scope.

• Identifies business requirements.

• Identifies major use cases.

• Estimates project cost and feasibility.

2. Elaboration Phase

• Analyzes requirements in detail.

• Establishes the system architecture.

• Identifies and reduces major risks.

• Creates the architectural foundation.

3. Construction Phase

• Actual software development takes place.

• Coding and integration are performed.

• Different system components are developed.

• Testing is performed.

4. Transition Phase

• Software is delivered to users.

• User acceptance testing is performed.

• Installation and deployment take place.

• User feedback and necessary corrections are handled.

Advantages

• Supports iterative development.

• Reduces project risks.

• Improves software quality.

• Handles changing requirements.

• Encourages continuous testing.

Applications

• Large software projects.

• Enterprise applications.

• Banking systems.

• Web-based applications.

Exam tip: Unified Methodology = Use cases + Architecture + Iterative development.

6. Rumbaugh Methodology (Object Modeling Technique – OMT)

Definition

Rumbaugh Methodology was developed by James Rumbaugh and his colleagues.

It is also called **Object Modeling Technique (OMT)**.

It is an object-oriented methodology used for analyzing, designing and implementing software systems using objects and their relationships.

It describes a system using three different models.

Three Models of OMT

• Object Model

• Dynamic Model

• Functional Model

1. Object Model

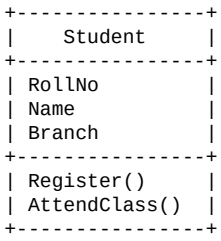
• Represents the static structure of a system.

• Identifies objects and classes.

• Shows attributes, operations and relationships.

• Usually represented using object diagrams or class diagrams.

Example:



2. Dynamic Model

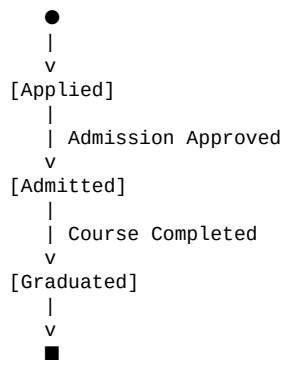
• Represents the behavior of a system over time.

• Describes states and transitions.

• Shows how objects respond to events.

• Uses State Transition Diagrams.

Example:



3. Functional Model

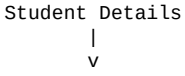
• Represents data processing and transformations.

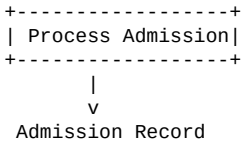
• Explains how input data is converted into output.

• Uses Data Flow Diagrams (DFDs).

• Shows processes and data movement.

Example:





Steps in Rumbaugh Methodology

• Develop the Object Model.

• Develop the Dynamic Model.

• Develop the Functional Model.

• Integrate all three models.

• Implement the system.

Advantages

• Provides a clear representation of a system.

• Separates static and dynamic behavior.

• Supports complex systems.

• Helps identify objects and relationships.

Applications

• Real-time systems.

• Banking systems.

• Information management systems.

• Industrial applications.

Exam tip: Rumbaugh = OMT = Object Model + Dynamic Model + Functional Model.

7. Booch Methodology

Definition

Booch Methodology was developed by Grady Booch.

It is an object-oriented software development methodology used for designing and developing complex software systems.

It focuses on identifying objects, classes, relationships and interactions between different system components.

Important Concepts

• **Objects:** Represent real-world entities.

• **Classes:** Define the properties and behavior of objects.

• **Encapsulation:** Combines data and methods into a single unit.

• **Inheritance:** Allows one class to acquire properties of another class.

• **Polymorphism:** Allows the same operation to behave differently.

• **Relationships:** Represent connections between objects.

Two Development Processes

Booch Methodology uses two major processes:

1. Macro Development Process

It represents the overall software development lifecycle.

Main activities:

• Conceptualization.

• Analysis and development.

• Design and evolution.

• Maintenance.

2. Micro Development Process

It represents the development activities performed within each iteration.

Main activities:

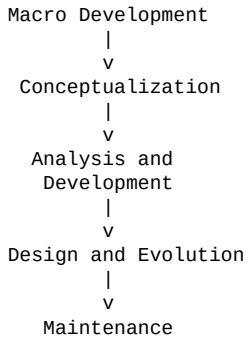
• Identify classes and objects.

• Identify their semantics.

• Identify relationships.

• Specify interfaces and implementation.

Booch Methodology Diagram



Steps in Booch Methodology

• Identify important classes and objects.

• Define their attributes and operations.

• Identify relationships between objects.

• Design the system architecture.

• Implement and test the system.

Advantages

• Suitable for complex software systems.

• Supports object-oriented design.

• Encourages reusability.

• Improves modularity.

• Supports iterative and incremental development.

Applications

• Large-scale software systems.

• Real-time applications.

• Distributed applications.

• Enterprise software.

Difference Between Unified, Rumbaugh and Booch Methodologies

Unified Methodology	Rumbaugh Methodology	Booch Methodology
Associated with Unified Process.	Developed by James Rumbaugh.	Developed by Grady Booch.
Uses use cases and iterative development.	Uses three models.	Uses macro and micro development processes.
Architecture-centric.	Model-centric.	Object-oriented design-centric.
Uses UML.	Uses OMT.	Uses Booch notation and design concepts.
Suitable for iterative development.	Suitable for modeling complex systems.	Suitable for designing complex systems.

Important: Rumbaugh, Booch and Jacobson's Objectory methodology contributed to the development of UML. UML was standardized by the OMG.

PART 3: SOFTWARE TESTING STRATEGIES

This is an important section because it contains several testing concepts and techniques.

8. A Strategic Approach to Software Testing

Definition

Software Testing is the process of executing software to identify errors, defects and failures and to ensure that the software meets specified requirements.

A Strategic Approach to Software Testing provides a systematic plan for testing software at different development stages.

Objectives of Software Testing

• Identify software defects.

• Verify that software meets requirements.

• Improve software quality.

• Ensure reliability and performance.

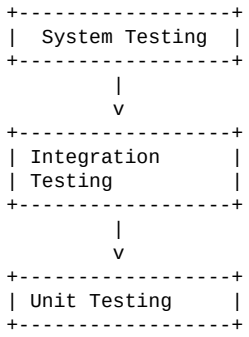
• Reduce maintenance costs.

• Provide confidence before software delivery.

Levels of Software Testing

Software testing is generally performed at different levels.

Diagram:



1. Unit Testing

• Individual modules or components are tested separately.

• Usually performed by developers.

• Checks the correctness of individual functions.

Example: Testing a login function independently.

2. Integration Testing

• Different modules are combined and tested.

• Checks communication between modules.

• Identifies interface-related errors.

Example: Testing whether the login module communicates correctly with the database.

3. Validation Testing

• Checks whether the developed software satisfies customer requirements.

• Ensures that the correct product has been developed.

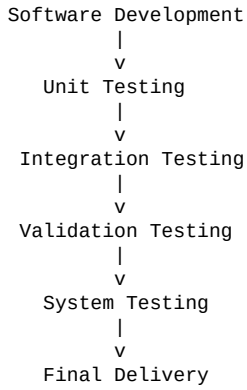
4. System Testing

• Tests the complete integrated software system.

• Checks overall functionality and performance.

• Includes security, recovery and stress testing.

General Testing Strategy



Important Principles

• Testing should begin early.

• Testing should be planned before execution.

• Test cases should be designed for expected and unexpected inputs.

• Testing should focus on high-risk areas.

• Independent testing improves defect detection.

• Exhaustive testing is generally impossible for non-trivial software.

Advantages

• Improves software quality.

• Reduces software failures.

• Increases customer satisfaction.

• Identifies defects early.

• Improves reliability.

9. Test Strategies for Conventional Software

Definition

Conventional Software Testing Strategy is an approach used to test traditional procedural or function-oriented software.

In conventional software, testing generally starts with individual modules and gradually moves towards the complete system.

Testing Approaches

1. Unit Testing

• Individual functions and modules are tested.

• Checks internal logic and correctness.

• Identifies coding errors.

2. Integration Testing

• Combines different modules.

• Tests communication between modules.

• Identifies interface errors.

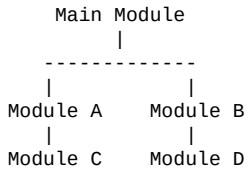
Two common integration approaches are:

Top-Down Integration Testing

• Testing begins with the top-level module.

• Lower-level modules are gradually integrated.

• Stubs are used when lower-level modules are unavailable.

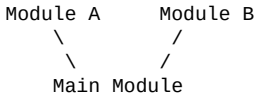


Bottom-Up Integration Testing

• Testing begins with lower-level modules.

• Higher-level modules are gradually integrated.

• Drivers are used to test lower-level modules.



3. Validation Testing

• Checks whether the software meets customer requirements.

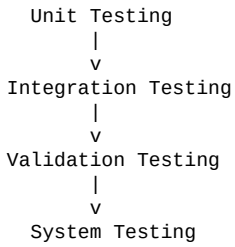
• Ensures that the system performs the required functions.

4. System Testing

• Tests the entire integrated system.

• Checks performance, security and reliability.

Conventional Testing Flow



Advantages

• Follows a systematic approach.

• Detects errors at different levels.

• Supports modular development.

• Helps improve reliability.

10. Test Strategies for Object-Oriented Software

Definition

Object-Oriented Software Testing is the process of testing software developed using object-oriented concepts such as classes, objects, inheritance, encapsulation and polymorphism.

Testing object-oriented software is different from conventional software because objects interact with one another and maintain their own states.

Testing Levels

1. Class Testing

• Individual classes are tested.

• Attributes and methods are checked.

• Verifies the behavior of individual objects.

Example: Testing a Student class and its methods.

2. Integration Testing

• Tests interactions between different classes.

• Checks communication between objects.

• Identifies errors in object relationships.

3. System Testing

• Tests the complete object-oriented application.

• Checks functional and non-functional requirements.

Important Testing Concepts

• **Class Testing:** Tests individual classes and their methods.

• **State-Based Testing:** Checks how objects change states.

• **Inheritance Testing:** Checks whether inherited methods work correctly.

• **Polymorphism Testing:** Checks whether overridden methods behave correctly.

• **Interaction Testing:** Checks communication between objects.

Example

Consider an online shopping application containing:

• Customer Class

• Product Class

• Order Class

• Payment Class

Testing checks:

• Whether Customer objects can place orders.

• Whether Order objects correctly access Product details.

• Whether Payment objects process payments correctly.

• Whether different classes communicate properly.

Difference Between Conventional and Object-Oriented Testing

Conventional Testing	Object-Oriented Testing
Focuses on functions and procedures.	Focuses on classes and objects.
Unit testing checks individual functions.	Class testing checks individual classes.
Focuses on control flow and data flow.	Focuses on object states and interactions.
Integration checks module communication.	Integration checks object communication.
Does not generally involve inheritance.	Must consider inheritance and polymorphism.

11. Validation Testing

Definition

Validation Testing is the process of checking whether the developed software satisfies the customer's requirements and intended purpose.

It answers the question:

"Are we building the right product?"

Objectives

• Verify customer requirements.

• Ensure software performs the expected functions.

• Identify missing requirements.

• Check user expectations.

• Ensure software is ready for delivery.

Characteristics

• Performed after integration testing.

• Focuses on customer requirements.

• Checks functional and non-functional requirements.

• Often involves acceptance testing.

• Ensures the software is suitable for its intended use.

Types of Acceptance Testing

1. Alpha Testing

• Conducted at the developer's location.

• Performed by selected users or testers.

• Helps identify defects before public release.

2. Beta Testing

• Conducted by actual users in a real environment.

• Users provide feedback.

• Helps identify real-world issues.

Example

Consider a college management application.

The customer expects:

• Student registration.

• Attendance management.

• Fee payment.

• Examination results.

Validation testing checks whether all these requirements are correctly implemented.

Advantages

• Improves customer satisfaction.

• Ensures requirements are fulfilled.

• Reduces the risk of delivering incorrect software.

• Helps identify missing functionality.

Verification vs Validation

Verification	Validation
Checks whether the software is built correctly.	Checks whether the correct software is built.
Focuses on specifications and design.	Focuses on customer requirements.
Often involves reviews and inspections.	Involves execution-based testing.
Are we building the product right?	Are we building the right product?

12. White-Box Testing

Definition

White-Box Testing is a software testing technique in which the internal structure, code, logic and control flow of a program are examined.

The tester has knowledge of the internal implementation of the software.

It is also called:

• Structural Testing.

• Glass-Box Testing.

• Clear-Box Testing.

Important Techniques

• Statement Coverage.

• Branch Coverage.

• Condition Coverage.

• Path Coverage.

• Loop Testing.

• Basis Path Testing.

1. Statement Coverage

Checks whether every statement in the program is executed at least once.

2. Branch Coverage

Checks whether every decision branch (True and False) is executed at least once.

3. Condition Coverage

Checks whether each individual Boolean condition evaluates to both True and False.

4. Path Coverage

Checks different possible execution paths through the program.

5. Loop Testing

Checks loops for:

• Zero iterations.

• One iteration.

• Multiple iterations.

• Boundary conditions.

Example

Consider the following code:

```
int a = 10;
int b = 20;

if (a < b) {
    System.out.println("A is smaller");
} else {
    System.out.println("B is smaller");
}
```

White-box testing examines:

• Whether both variables are initialized.

• Whether the condition is evaluated correctly.

• Whether the True branch executes.

• Whether the False branch executes.

Advantages

• Helps identify logical errors.

• Finds hidden code defects.

• Improves code quality.

• Helps measure statement and branch coverage.

Disadvantages

• Requires programming knowledge.

• Time-consuming for large applications.

• Exhaustive path testing may be impossible.

Exam tip: White-Box Testing = Testing internal code and logic.

13. Basis Path Testing

This is one of the most important topics in your syllabus because it can be asked as a theory question or a numerical problem.

Definition

Basis Path Testing is a white-box testing technique introduced by Thomas McCabe.

It identifies a set of linearly independent execution paths in a program.

These paths are selected using the **Cyclomatic Complexity** of the program.

Important Terms

1. Control Flow Graph (CFG)

A Control Flow Graph represents the flow of execution in a program using nodes and edges.

• Node: Represents a statement or block of statements.

• Edge: Represents the flow of control between nodes.

2. Independent Path

An independent path is a path that introduces at least one new edge that has not been included in previously selected paths.

3. Cyclomatic Complexity

Cyclomatic Complexity measures the number of linearly independent paths in a program.

Formula for Cyclomatic Complexity

There are three important formulas:

Formula 1:

$$
V(G)=E-N+2
$$

Where:

• E = Number of edges.

• N = Number of nodes.

Formula 2:

$$
V(G)=P+1
$$

Where:

• P = Number of predicate (decision) nodes.

Formula 3:

$$
V(G)=\text{Number of Regions}
$$

Example

Consider the following program:

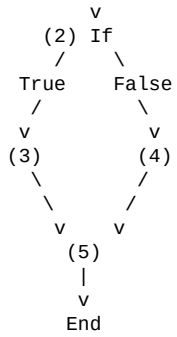
```
int a = 10;
int b = 20;

if (a < b) {
    System.out.println("A is smaller");
} else {
    System.out.println("B is smaller");
}

System.out.println("End");
```

Control Flow Graph





Here:

• Nodes (N) = 5

• Edges (E) = 6

• Predicate nodes (P) = 1

Calculate Cyclomatic Complexity

Using Formula 1:

$$
V(G)=E-N+2
$$

$$
V(G)=6-5+2
$$

$$
\boxed{V(G)=3}
$$

Correction to note: In this simple if-else graph, the two branches merge into a common final node. The graph has 5 nodes and 6 edges only if the final node is counted separately as shown, giving 3. However, the standard cyclomatic complexity for one binary decision is 2. The correct CFG for the program has 5 nodes and 5 edges if the common ending node is counted, so:

$$
V(G)=5-5+2=2
$$

Using Formula 2:

$$
V(G)=P+1
$$

$$
V(G)=1+1
$$

$$
\boxed{V(G)=2}
$$

Therefore, the program has **2 independent paths**.

Independent Paths

Path 1:

```
1 → 2 → 3 → 5
```

Condition is True.

Path 2:

```
1 → 2 → 4 → 5
```

Condition is False.

Steps in Basis Path Testing

- Draw the Control Flow Graph.
- Identify the number of nodes and edges.
- Calculate Cyclomatic Complexity.
- Identify independent paths.
- Design test cases for each independent path.
- Execute the test cases.

Advantages

- Identifies independent execution paths.
- Helps measure program complexity.
- Improves test coverage.
- Detects logical errors.
- Provides a systematic testing approach.

Very Important for Exams: Remember these formulas:

$$
\boxed{V(G)=E-N+2}
$$

\]

\[
\boxed{V(G)=P+1}
\]

\[
\boxed{V(G)=\text{Number of Regions}}
\]

14. Black-Box Testing

Definition

Black-Box Testing is a software testing technique in which the functionality of a system is tested without knowing its internal code or implementation.

The tester provides inputs and checks whether the outputs match the expected results.

It is also called:

- Functional Testing.
- Behavioral Testing.
- Specification-Based Testing.

Important Techniques

1. Equivalence Partitioning

In this technique, input data is divided into different groups called equivalence classes.

Each group contains inputs that are expected to behave similarly.

Example:

Suppose an application accepts age between 18 and 60.

Valid input:

\[
18 \leq Age \leq 60
\]

Invalid inputs:

\[

Age < 18

\]

\[

Age > 60

\]

Instead of testing every possible age, we select representative values from each group.

Partition	Example Input	Expected Result
Below 18	15	Invalid
18–60	30	Valid
Above 60	65	Invalid

2. Boundary Value Analysis

This technique tests values at the boundaries of input ranges because errors frequently occur at boundary conditions.

For an age range of 18 to 60:

Test Input	Expected Result
17	Invalid
18	Valid
19	Valid
59	Valid
60	Valid
61	Invalid

3. Decision Table Testing

Used when the output depends on multiple conditions.

Example: Login system.

Username	Password	Result
Valid	Valid	Login Success
Valid	Invalid	Login Failed
Invalid	Valid	Login Failed
Invalid	Invalid	Login Failed

4. State Transition Testing

Tests how the system behaves when it moves from one state to another.

Example: An ATM card gets blocked after three incorrect PIN attempts.

Advantages

• No programming knowledge required.

• Tests software from the user's perspective.

• Useful for functional testing.

• Helps identify missing functionality.

Disadvantages

• Internal code errors may remain undetected.

• Complete input coverage is difficult.

• Test cases depend heavily on requirements.

White-Box vs Black-Box Testing

White-Box Testing	Black-Box Testing
Tests internal code.	Tests external functionality.
Requires programming knowledge.	Does not require programming knowledge.
Focuses on code structure.	Focuses on input and output.
Tests statements and branches.	Tests requirements and behavior.
Example: Path Coverage.	Example: Boundary Value Analysis.

Exam tip: White-Box = Internal working. Black-Box = External functionality.

15. System Testing

Definition

System Testing is the process of testing a complete and integrated software system to verify that it meets specified functional and non-functional requirements.

It is generally performed after integration testing.

Objectives

• Verify the complete system.

• Identify system-level defects.

• Check performance and reliability.

• Verify security.

• Ensure the software meets requirements.

Types of System Testing

There are several important types.

1. Recovery Testing

• Checks whether the system can recover after failures.

• Tests backup and restoration mechanisms.

• Example: Recovering data after a server crash.

2. Security Testing

• Checks whether the system is protected against unauthorized access.

• Tests authentication and authorization.

• Example: Checking whether unauthorized users can access admin pages.

3. Stress Testing

• Checks system behavior under extreme workload conditions.

• Identifies the breaking point of the system.

• Example: Testing a website with an extremely large number of simultaneous users.

4. Performance Testing

• Checks speed, response time and resource utilization.

• Ensures the system performs efficiently.

• Example: Checking whether a website loads within an acceptable time.

5. Load Testing

• Checks system performance under expected workload conditions.

• Example: Testing an application with 5,000 concurrent users.

6. Deployment Testing

• Checks whether software works correctly in its target environment.

• Verifies installation and configuration.

Advantages

• Tests the entire application.

• Identifies system-level defects.

• Ensures reliability and security.

• Improves user satisfaction.

• Helps verify production readiness.

IMPORTANT DIFFERENCES FOR SEMESTER EXAM

These comparisons are frequently useful for 6-mark questions.

1. Activity Diagram vs State Chart Diagram

Activity Diagram	State Chart Diagram
Represents workflow.	Represents object states.
Focuses on activities.	Focuses on state changes.
Shows sequence of operations.	Shows transitions caused by events.
Example: ATM withdrawal.	Example: ATM card states.

2. Component Diagram vs Deployment Diagram

Component Diagram	Deployment Diagram
Represents software components.	Represents hardware nodes.
Shows software organization.	Shows physical deployment.
Focuses on modules.	Focuses on devices and servers.
Example: Payment Module.	Example: Database Server.

3. White-Box vs Black-Box Testing

White-Box Testing	Black-Box Testing
Internal code is examined.	Internal code is not examined.
Requires programming knowledge.	Does not require programming knowledge.
Tests logic and control flow.	Tests functionality.
Statement and branch coverage.	Equivalence partitioning and boundary value analysis.

4. Conventional vs Object-Oriented Testing

Conventional	Object-Oriented
Function-oriented.	Class-oriented.
Tests individual functions.	Tests individual classes.
Focuses on control flow.	Focuses on object behavior.
Module integration.	Object interaction.

5. Verification vs Validation

Verification	Validation
Are we building the product right?	Are we building the right product?
Checks specifications and design.	Checks customer requirements.
Reviews and inspections.	Execution-based testing.

UNIT 4 – IMPORTANT QUESTIONS FOR EXAM

Based on your syllabus, prepare these questions first.

Priority	Important Question	Marks
■■■	Explain Activity Diagram with a suitable example.	6
■■■	Explain State Chart Diagram with an example.	6
■■■	Explain Component and Deployment Diagrams with examples.	6
■■■	Explain Rumbaugh Methodology and its three models.	6
■■■	Explain Unified Methodology and its phases.	6
■■■	Explain Booch Methodology.	6
■■■	Explain the strategic approach to software testing.	6
■■■	Explain White-Box Testing and its techniques.	6
■■■	Explain Basis Path Testing with an example.	6
■■■	Explain Black-Box Testing and its techniques.	6
■■■	Explain different types of System Testing.	6
■■	Explain Validation Testing and its types.	6
■■	Compare conventional and object-oriented testing strategies.	6
■■	Differentiate White-Box and Black-Box Testing.	6

Last-Minute Revision – Remember These Points

Topic	What to Remember
Activity Diagram	Workflow of activities
State Chart Diagram	States and transitions
Component Diagram	Software modules
Deployment Diagram	Hardware and deployment
Unified Methodology	Inception, Elaboration, Construction, Transition
Rumbaugh Methodology	Object, Dynamic, Functional Models

Topic	What to Remember
Booch Methodology	Macro and Micro processes
Software Testing Strategy	Unit → Integration → Validation → System
Conventional Testing	Function and module testing
Object-Oriented Testing	Class and object interaction testing
Validation Testing	Are we building the right product?
White-Box Testing	Internal code testing
Basis Path Testing	Independent paths and Cyclomatic Complexity
Black-Box Testing	Input and output testing
System Testing	Testing the complete system

My recommendation for your preparation

Give special attention to these five topics:

• **Rumbaugh Methodology** – Three models are very important.

• **UML Diagrams** – Practice drawing all four diagrams.

• **Basis Path Testing** – Understand the CFG and practice Cyclomatic Complexity numericals.

• **White-Box vs Black-Box Testing** – Prepare definitions, techniques and differences.

• **System Testing** – Remember all six types with examples.

Important: Don't just memorize the diagrams. Practice drawing them once or twice yourself. In a 6-mark Software Engineering answer, a correct diagram, definition, 4–5 explanation points and a suitable example will make your answer much stronger.